Performance Testing

Date	1 July 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	VS Code,Postman,
Type of Testing	Load Testing, Stress Testing, Spike Testing
Target Module / API	HDI Prediction API (/predict)
Test Environment	Local Flask Server (Windows 11, Python 3.11)
Test Date	1 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal prediction requests	10	60	All requests processed successfully
2	Concurrent prediction requests	50	120	Response time remains < 2 sec
3	Stress testing with high load	100	180	System remains stable with minimal errors
4	Spike testing sudden traffic increase	150	30	API recovers quickly without crashing

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.84 sec	PASS	Within acceptable limit
2	Response Time (Max)	< 5 seconds	2.91 sec	PASS	No timeout observed
3	Throughput (Req/sec)	20 req/sec	28.4 req/sec	PASS	Good processing capacity
4	Error Rate	< 1%	0.3%	PASS	Very low failure rate
5	CPU Utilization	< 80%	64%	PASS	Stable under load
6	Memory Utilization	< 80%	58%	PASS	Memory usage acceptable

Step 4: Observations & Analysis

Key Findings

- The HDI Prediction System handled concurrent prediction requests efficiently.
- Average prediction response time remained below 2 seconds.
- No prediction failures or server crashes occurred during testing.
- The Flask application remained stable under moderate workloads.
- The Random Forest model produced accurate and consistent HDI predictions.

Bottlenecks Identified

- Response time increased slightly under very high user load.
- Initial model loading caused a small delay for the first prediction.
- Interactive charts slightly increased page loading time.

Optimization Steps Taken

- Optimized Flask routing and API request handling.
- Reduced unnecessary preprocessing operations.
- Loaded the trained Random Forest model once during startup.
- Optimized NumPy/Pandas processing for faster predictions.
- Improved frontend validation and AJAX-based result updates.
- Optimized Chart.js rendering for better performance.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):